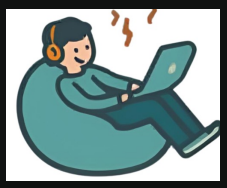

The Lazy Developer

21

MODULE

Building Custom Skills for Claude Code

Learn how to create and use custom skills with Claude Code to automate repetitive tasks, enforce coding standards, and build reusable workflows that supercharge your development process.

Table of Contents

What Are Claude Skills and Why They Matter

Let's imagine you hire a brilliant new assistant. On day one, they can do just about anything you ask from things like write emails, organise files, and researching competitors. The trouble is you have to explain exactly what you want every single time. "Draft me an email to the supplier. Use a professional tone. Include the order number from the spreadsheet. Sign off with my name." Every. Single. Time.

Now imagine you could hand that assistant a laminated card that says: "When I say 'supplier email,' follow these steps." Suddenly, one short phrase triggers a perfect, repeatable action. No re-explaining. No forgetting steps. No drift in quality.

That laminated card? That is essentially what a Claude Code skill is.

Skills in Plain English

A Claude Code skill is a reusable prompt template saved as a Markdown file. Instead of typing out a long, detailed instruction every time you want Claude Code to do something, you write that instruction once, save it in a special folder, and then invoke it with a simple slash command like /commit or /review-pr.

Here is the core idea:

Without a Skill	With a Skill
You type a 15-line prompt explaining exactly how to write a commit message every time you commit	You type /commit and the skill handles the rest
You copy-paste your code review checklist from a notes app into the chat	You type /review-pr and a thorough, consistent review runs automatically
You forget to check for security issues during reviews because you are tired	The skill always checks because the instructions are baked in
Different team members get wildly different outputs from Claude	Everyone shares the same skill file, so outputs are consistent

At their simplest, skills are just Markdown files with a bit of frontmatter at the top (think of frontmatter as a label on the outside of an envelope—it tells Claude Code what the skill is called and when to use it). The body of the file contains the actual prompt instructions.

How Skills Differ from Just Asking Claude Directly

If you have been using Claude Code for a while (or Cursor, or any AI coding tool), you might be thinking: "I already ask Claude to do these things. Why do I need a skill?"

Great question. Here is the difference:

Asking Claude directly is like cooking a meal from memory. Sometimes you nail it, sometimes you forget the paprika, sometimes you accidentally double the salt because you were distracted.

Using a skill is like cooking from a tested recipe card. The ingredients and steps are written down. You get the same great result whether you are cooking at 10 a.m. on a Saturday or 11 p.m. after a long day.

Let's break that down more concretely:

Aspect	Ad-hoc Prompting	Using a Skill
Consistency	Varies by mood, memory, and how much coffee you have had	Identical instructions every time
Speed	You re-type or rephrase the same instructions repeatedly	One slash command triggers everything
Shareability	Locked in your head or a random note somewhere	A file in your repo that anyone on the team can use
Version control	None—your prompts vanish into chat history	The skill file lives in Git, so you can track changes over time
Complexity	Hard to maintain a 50-line prompt in your head	A 50-line skill file is perfectly manageable
Tool access	You have to remember to tell Claude which tools to use	The skill can specify exactly which tools (Bash, Grep, Read, Edit, etc.) to leverage

The bottom line: skills turn your best one-off prompts into reliable, repeatable workflows.

The Skill Ecosystem: Where Do Skills Live?

Claude Code looks for skills in two places:

1. Project skills — stored in your project's .claude/skills/ directory. These travel with your codebase and are shared with anyone who clones the repo.
2. Personal skills — stored in your home directory at ~/.claude/skills/. These are private to you and available across all your projects.

Think of it like this:

- Project skills = the office playbook. Everyone on the team follows the same procedures.
- Personal skills = your personal shortcuts. Maybe you like a particular way of formatting TODO comments, or you always want Claude to greet you with a terrible pun. That is your business.

```
Your Project/
% % %  .claude/
%    % % %  skills/
%        % % %  commit.md          # Team-shared commit message skill
%        % % %  review-pr.md       # Team-shared code review skill
%        % % %  write-tests.md     # Team-shared test writing skill
% % %  src/
% % %  package.json
% % %  ...

Home Directory/
% % %  ~/.claude/
    % % %  skills/
        % % %  my-todo-style.md   # Personal preference
        % % %  daily-standup.md   # Personal daily workflow
```

When you type a slash command in Claude Code, it checks both locations. Project skills take precedence if there is a name collision.

Why Skills Matter for Non-Coders Especially

If you are a non-coder building with AI tools, skills are arguably more valuable to you than to experienced developers. Here is why:

1. They encode expertise you do not have yet.
A well-written skill for code reviews can check for security vulnerabilities, performance issues, and best practices—things you might not know to look for yourself. The skill becomes your safety net.

2. They reduce decision fatigue.
When you are learning to build software, every step feels like a decision. "Should I commit now? What should the message say? Did I break anything?" A skill removes that mental overhead. You just type the command and trust the process.

3. They help you learn.
Because skill files are just Markdown, you can read them. You can see exactly what steps a good code review involves, or what makes a great commit message. Over time, the patterns sink in and you start understanding the "why" behind the "what."

4. They make you look like a pro.
When your commit messages are consistently well-formatted, your code reviews are thorough, and your documentation is always up to date—people notice. Skills help you punch above your weight from day one.

How Skills Compare Across AI Coding Tools

Claude Code is not the only AI coding tool with this concept. Let's see how the idea of reusable prompts shows up across the landscape:

Tool	Equivalent Feature	How It Works
Claude Code	Skills (.claude/skills/)	Markdown files with frontmatter, invoked via slash commands
Cursor	Rules for AI (.cursorrules)	A single file that sets global instructions for all AI interactions in the project
Cursor	Notepads	Reusable context snippets you can attach to any chat message
GitHub Copilot	Custom Instructions (.github/copilot-instructions.md)	A Markdown file that guides Copilot's behaviour across the repo
Lovable	Project Settings / AI Instructions	A text area in the UI where you can set persistent instructions
Replit	.replit config + AI chat	Configuration file plus in-chat instructions

The pattern is clear: every major AI coding tool is moving toward letting you save and reuse your best prompts. Claude Code's skill system is one of the most flexible implementations because each skill is its own file with its own trigger, rather than one monolithic instruction document.

What Can a Skill Actually Do?

Skills have access to all of Claude Code's built-in tools. That means a single skill can:

- Read files in your project (using the Read tool)
- Search your codebase for patterns (using Grep and Glob)
- Run terminal commands (using Bash)
- Edit files directly (using Edit and Write)
- Create new files when needed
- Analyse code and provide recommendations
- Generate documentation based on your actual codebase

This is what makes skills so powerful. They are not just text templates but rather they are action templates. A skill can read your staged git changes, analyse what you changed, check for common issues, and then write a perfectly formatted commit message. All from a single /commit command.

A Quick Taste: What a Skill File Looks Like

Before we dive into building skills in the next section, here is a sneak peek at what a skill file looks like. Don't worry about understanding every line—we will break it all down step by step.

```
---
name: commit
description: Generate a well-formatted commit message
---

Analyse all staged changes using `git diff --cached` and create a commit message
that follows conventional commit format.

Rules:
- Start with a type: feat, fix, docs, style, refactor, test, or chore
- Keep the subject line under 72 characters
- Add a blank line then a body explaining WHY the change was made
- If there are multiple logical changes, suggest splitting into separate commits

After generating the message, show it to the user and ask for confirmation
before running `git commit`.
```

That is it. A bit of frontmatter (the part between the --- lines) and then plain English instructions. No code to compile. No APIs to configure. No package to install. Just a Markdown file.

Get Claude to write the skills

That's right, you didn't think you were going to have to come up with skills yourself did you? There's plenty out there on the internet already, HOWEVER, you should always review a skill before installing it because there are some reports out there that say that some skills are malicious with prompt injection attacks. Some text that might give an instruction to Claude to do somethig you didn't know about like share an API key. So just be a little careful.

Otherwise, just write your own for how you think you want a skill to look like. Is it designing the landing page to a specific style, or NOT style it to? Is it how to comment in code, organise files, you name it. Skills can be for anything.

Built-in Skills vs Custom Skills

Claude Code ships with a few built-in skills out of the box:

- /commit — Generates commit messages from your staged changes
- /review-pr — Reviews a pull request for issues and improvements

These are great starting points, but the real magic happens when you start building your own skills tailored to your specific project, your team's conventions, and your personal workflow. That is exactly what we are going to do in the next section.

Key Takeaways

- Claude Code skills are reusable prompt templates saved as Markdown files that you invoke with slash commands.
- They live in two places: .claude/skills/ in your project (shared with the team) or ~/.claude/skills/ in your home directory (personal).
- Skills differ from ad-hoc prompting because they are consistent, shareable, version-controlled, and repeatable.
- Every major AI coding tool is moving toward reusable prompts—Claude Code's file-per-skill approach is one of the most flexible.
- Skills have access to all of Claude Code's tools, so they can read, search, edit, and run commands—not just generate text.
- For non-coders, skills are a force multiplier: they encode expertise you are still building, reduce decision fatigue, and help you produce professional-quality output from day one.
- The productivity gains compound across every task you automate—commit messages, reviews, tests, docs, and more.

Getting Started: Your First Custom Skill

Remember the first time you made a spreadsheet formula work? Maybe it was a simple =SUM(A1:A10) and you felt like a wizard. Building your first Claude Code skill is going to feel like that except instead of adding up numbers, you are teaching an AI assistant a brand-new trick that it will perform flawlessly every time you ask.

In this section, we are going to build a custom skill from scratch. Step by step. No prior experience required. By the end, you will have a working skill that you can invoke with a slash command and start using immediately.

Before We Start: What You Need

To follow along, you will need:

1. Claude Code installed and working on your machine (if you have been following this course, you are already set)
2. A project folder — any project will do, even a simple one with a few files
3. A terminal open in your project directory
4. A text editor — VS Code, Cursor, or even the basic text editor on your computer

That is it. No packages to install. No configuration files to wrestle with. Skills are just Markdown files, and you already know more about Markdown than you think (it is the same formatting used in GitHub READMEs, Notion pages, and chat messages with bold and italic text).

Step 1: Create the Skills Directory

First, we need to create the folder where Claude Code looks for project skills. Open your terminal and navigate to your project root, then run:

```
mkdir -p .claude/skills
```

The -p flag means "create parent directories if they do not exist." So if you do not have a .claude folder yet, this command creates both .claude and .claude/skills in one go.

Your project structure should now look something like this:

```
my-project/
% % %  .claude/
%    % % %  skills/          # This is where your skills will live
% % %  src/
% % %  package.json
% % %  ...
```

> *Tip for Cursor users: If you are more comfortable working in Cursor or VS Code, you can just right-click in the file explorer and create these folders manually. The result is the same.*

Step 2: Choose What Your Skill Will Do

Before writing a single line, ask yourself: What task do I do repeatedly that I wish I could automate?

Here are some ideas to get the juices flowing:

Repetitive Task	Potential Skill Name
Writing commit messages	/commit
Adding comments to confusing code	/explain
Creating a new component file with boilerplate	/new-component
Checking for TODO comments across the codebase	/todos
Generating a summary of recent changes	/changelog
Running a pre-deployment checklist	/deploy-check

For this tutorial, we are going to build a changelog generator skill. It is practical, useful for any project, and demonstrates several key skill features without being too complex.

Our skill will:

- Look at recent git commits
- Group them by type (feature, fix, docs, etc.)
- Generate a nicely formatted changelog entry
- Ask before writing anything to a file

Step 3: Create the Skill File

Create a new file called changelog.md inside your .claude/skills/ directory:

```
touch .claude/skills/changelog.md
```

Or, if you prefer, just create it in your text editor. The file name matters as it becomes part of how you reference the skill. Keep it short, descriptive, and lowercase.

Now open that file and let's start writing.

Step 4: Write the Frontmatter

Every skill file starts with frontmatter which a small block at the top of the file between two lines of triple dashes (---). Think of frontmatter as the label on a filing folder. It tells Claude Code the skill's name and what it does.

```
---
name: changelog
description: Generate a formatted changelog entry from recent git commits
---
```

Let's break that down:

Field	What It Does	Example
name	The slash command name (what you type to invoke it)	changelog means you will type /changelog
description	A short summary shown in the skill list	Helps you remember what the skill does

That is the entire frontmatter. Two fields. Done.

Step 5: Write the Skill Instructions

Below the frontmatter, you write the actual instructions in plain English (or whatever language you prefer). This is the prompt that Claude Code will follow when you invoke the skill.

Here is our complete changelog skill:

```
---
name: changelog
description: Generate a formatted changelog entry from recent git commits
---

Generate a changelog entry based on recent git activity. Follow these steps:

## Step 1: Gather Recent Commits

Run `git log --oneline --no-merges -20` to get the last 20 non-merge commits.
If the user specifies a date range or number of commits, adjust the command
accordingly.

## Step 2: Categorise the Commits

Group each commit into one of these categories based on its message:

- **Features** - New functionality (commits starting with feat, add, new)
- **Bug Fixes** - Corrections (commits starting with fix, patch, hotfix)
- **Documentation** - Docs changes (commits starting with docs, readme)
- **Maintenance** - Refactoring, dependencies, configs (chore, refactor, deps)
- **Other** - Anything that does not fit the above

## Step 3: Format the Changelog

Create a changelog entry using this format:
```

[Date] - vX.X.X

Features

- Description of feature (commit hash)

Bug Fixes

- Description of fix (commit hash)

Documentation

- Description of docs change (commit hash)

Maintenance

- Description of maintenance task (commit hash)

```
Rules:
- Use today's date in YYYY-MM-DD format
- Rewrite commit messages to be user-friendly (remove prefixes, improve clarity)
- Include the short commit hash in parentheses for reference
- Skip empty categories (do not show "### Features" if there are none)
- Keep descriptions concise but meaningful

## Step 4: Present and Confirm

Show the generated changelog to the user. Ask if they would like to:
1. Append it to an existing CHANGELOG.md file
2. Create a new CHANGELOG.md file
3. Just copy it (do not write to any file)

Only write to a file after the user confirms.
```

Take a moment to notice a few things about this skill:

- It uses plain English. No programming syntax, no special API calls. Just clear instructions.
- It uses headings to organise steps. This helps Claude Code understand the workflow sequence.
- It specifies exact commands to run (git log --oneline --no-merges -20). The more specific you are, the more reliable the output.
- It includes formatting rules. This ensures consistent output every time.
- It asks for confirmation before writing. This is a great safety pattern as you never let a skill silently modify files without the user's okay.

Step 6: Test Your Skill

Save the file and open Claude Code in your project directory. Now type:

```
/changelog
```

Claude Code will find your skill file, read the instructions, and start executing them. It will run the git log command, categorise your commits, format the output, and ask you what to do with it.

If your project has git history, you should see a beautifully formatted changelog entry appear. If you are working in a brand-new project with few commits, that is fine too—the skill will work with whatever history is available.

Step 7: Iterate and Improve

Your first version will almost certainly not be perfect, and that is completely normal. Here is how to iterate:

Problem: The categories do not match your commit style.
Solution: Update the categorisation rules in the skill file to match how you actually write commits.

Problem: The output format is not quite right.
Solution: Adjust the format template in Step 3. Maybe you want bullet points instead of headers, or you want to include the full commit message.

Problem: 20 commits is too many (or too few).
Solution: Change the default number, or add a note that the user can specify a custom count.

The beauty of skills is that improving them is as simple as editing a text file. There is no recompilation, no redeployment, no restarting anything. Save the file, invoke the skill, see the result.

Understanding the Skill File Anatomy

Now that you have built one, let's formalise what we have learned about skill file structure:

```
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%   --- (frontmatter start)              %
%   name: skill-name                     %   !• Slash command trigger
%   description: What this skill does     %   !• Shown in skill list
%   --- (frontmatter end)                 %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %$
%                                        %
%   Skill instructions in plain English   %   !• The actual prompt
%   - Use headings for steps              %
%   - Include specific commands            %
%   - Set formatting rules                %
%   - Add safety checks                   %
%                                        %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
```

The body of the skill (everything after the frontmatter) supports full Markdown:

- Headings (##, ###) to organise steps
- Bold and italic for emphasis
- Code blocks for commands and templates
- Lists for rules and options
- Tables for structured information

Claude Code reads and interprets all of this formatting, which helps it understand the structure of your instructions.

Project Skills vs Personal Skills: A Quick Guide

Now that you know how to create a skill, you need to decide where to put it.

Question	If Yes !’	If No !’
Should teammates use this skill too?	Project skill (.claude/skills/)	Personal skill (~/.claude/skills/)
Is this specific to one codebase?	Project skill	Personal skill
Is this my personal preference or style?	Personal skill	Project skill
Should this be version-controlled with the project?	Project skill	Personal skill

To create a personal skill, just use the home directory path instead:

```
mkdir -p ~/.claude/skills
touch ~/.claude/skills/my-skill.md
```

Personal skills are available in every project you work on with Claude Code. They are great for things like your preferred commit message format, your personal code commenting style, or a daily standup helper.

Passing Arguments to Skills

Some skills benefit from taking input when you invoke them. For example, you might want to tell the changelog skill to look at a specific number of commits:

```
/changelog 10
```

Or provide a date range:

```
/changelog since last Monday
```

You do not need any special syntax in the skill file to support this. Claude Code is smart enough to take whatever text you type after the slash command and use it as context when following the skill instructions. In our changelog skill, we included the line:

> *"If the user specifies a date range or number of commits, adjust the command accordingly."*

That single sentence is enough for Claude Code to handle arguments gracefully. It will modify the git log command based on what you provide.

Your Second Skill: A Quick Exercise

Now that you have built one skill, try building a second one on your own. Here is a simple one to try—a TODO finder:

```
---
name: todos
description: Find and summarise all TODO comments in the codebase
---

Search the entire project for TODO, FIXME, HACK, and XXX comments in source
code files.

Steps:
1. Use Grep to search for the patterns: TODO, FIXME, HACK, XXX
2. Ignore node_modules, dist, build, and .git directories
3. Group results by file
4. For each TODO, show:
   - The file path and line number
   - The full comment text
   - A brief assessment: is this critical, nice-to-have, or outdated?
5. Provide a summary at the end:
   - Total count of each type
   - Top 3 most critical items to address
   - Any TODOs that look stale (in files not modified recently)

Format the output as a clean Markdown report.
```

Save this as .claude/skills/todos.md and test it with /todos. You now have two custom skills in your toolkit.

Common Mistakes to Avoid

As you start building skills, watch out for these pitfalls:

1. Being too vague.
Bad: "Review my code and fix issues."
Good: "Check all TypeScript files in /src for unused imports, variables declared but never read, and console.log statements that should be removed before production."

2. Forgetting safety checks.
Bad: A skill that deletes files without asking.
Good: A skill that lists what it would delete and asks for confirmation.

3. Making the skill too broad.
Bad: "Do everything needed to deploy the app."
Good: "Run the pre-deployment checklist: verify tests pass, check for console.logs, ensure environment variables are set, and confirm the build succeeds."

4. Not testing with edge cases.
What happens if there are no git commits? What if a search returns zero results? Good skills handle these gracefully.

What's Next

You have now built your first (and hopefully second) custom skill. In the next section, we will cover best practices for skill design so things like how to write skills that are robust, maintainable, and genuinely useful over the long term. We will also cover some clever patterns that experienced skill authors use to get the most out of their Claude Code workflows.

Key Takeaways

- Creating a skill is as simple as making a Markdown file in .claude/skills/ with frontmatter and plain English instructions.
- The mkdir -p .claude/skills command creates your skills directory in one step.
- Frontmatter only needs two fields: name (the slash command) and description (a short summary).
- Write instructions in plain English using headings, lists, and code blocks to organise steps.
- Always include a confirmation step before your skill writes to or modifies files.
- Test and iterate freely—editing a skill file takes effect immediately with no compilation or restart needed.
- Choose between project skills and personal skills based on whether the skill should be shared with your team or kept private.
- Arguments are handled naturally—just mention in your instructions that the user might provide additional context, and Claude Code will use it.
- Start simple and expand—your first skill does not need to be elaborate. A 10-line skill that saves you 5 minutes a day is worth its weight in gold.

Best Practices for Skill Design

Think about the best recipe you have ever followed. Not a Michelin-star, 47-ingredient extravaganzq, but the one that just works every time. The one where the instructions are clear, the portions are sensible, and the result is consistently good. That recipe was not great by accident. Someone tested it, refined it, and made deliberate choices about what to include and what to leave out.

Designing great Claude Code skills is exactly the same process. In the previous section, you built your first skill and got it running. Now we are going to level up. This section is about writing skills that are robust (they handle surprises), maintainable (you can update them without breaking things), and genuinely useful (you actually want to use them every day).

Principle 1: Scope It Like a Laser, Not a Floodlight

The single most common mistake in skill design is trying to do too much. A skill called "do-everything" that reviews code, writes tests, generates docs, and deploys your app sounds impressive on paper—but in practice, it will be slow, unpredictable, and hard to debug.

Great skills do one thing well.

Overly Broad Skill	Better: Focused Skills
/code-assistant that reviews, tests, and deploys	/review for code review, /write-tests for tests, /deploy-check for deployment
/fix-everything that finds and fixes all bugs	/lint-check for style issues, /security-scan for vulnerabilities
/setup-project that scaffolds everything from scratch	/new-component for components, /new-route for API routes, /new-test for test files

The rule of thumb: If you cannot describe what your skill does in one sentence without using the word "and," it is probably too broad. Split it up.

Why does this matter? Two reasons:

1. Reliability. The more steps a skill has, the more places it can fail. A 5-step skill is far more reliable than a 25-step one.
2. Composability. Small, focused skills can be used together. You might run /lint-check then /write-tests then /commit in sequence. Each one does its job cleanly.

Principle 2: Be Specific About Commands and Tools

Claude Code is remarkably good at interpreting vague instructions, but "remarkably good" is not the same as "perfectly reliable." The more specific you are, the more consistent your results.

Compare these two approaches:

Vague:

```
Check what files have changed and summarise the changes.
```

Specific:

```
Run `git diff --cached --stat` to see which files are staged for commit.
Then run `git diff --cached` to see the full diff.
Analyse the changes and provide a summary grouped by:
- Files modified
- Lines added vs removed
- Types of changes (new features, bug fixes, refactoring)
```

The specific version will produce consistent results across different projects, different days, and different moods of the AI. The vague version might use git status one time and git diff another, giving you different levels of detail.

Pro tip: When your skill needs to run a command, write the exact command you want. When it needs to search for files, specify the patterns. When it needs to read a file, name it (or describe how to find it).

Principle 3: Structure Your Instructions with Headings

Claude Code reads your skill file as Markdown, and it understands document structure. Use that to your advantage.

Flat, wall-of-text instructions (harder for Claude to follow):

```
Look at the staged changes and write a commit message. Use conventional
commit format. The subject should be under 72 characters. Add a body
explaining why. If there are breaking changes mention them. Check for
sensitive files like .env and warn about them. Show the message before
committing.
```

Structured instructions (easier for Claude to follow):

```
## Step 1: Analyse Staged Changes
Run `git diff --cached --stat` to identify what files are staged.
Run `git diff --cached` to see the full changes.

## Step 2: Check for Sensitive Files
If any staged files match these patterns, WARN the user and stop:
- .env, .env.*, *.key, *.pem
- credentials.json, secrets.*
- Any file containing API keys or passwords

## Step 3: Generate the Commit Message
Format: conventional commits (feat, fix, docs, chore, etc.)
Rules:
- Subject line under 72 characters
- Blank line between subject and body
- Body explains WHY the change was made, not just WHAT changed
- Mention breaking changes with a BREAKING CHANGE footer

## Step 4: Confirm and Commit
Show the generated message. Ask the user to confirm, edit, or cancel.
Only run `git commit` after explicit confirmation.
```

The structured version is not just easier for you to read, it is easier for Claude Code to follow reliably. Each step is clearly delineated, so the AI knows exactly when one phase ends and the next begins.

Principle 4: Always Include Safety Rails

This one is non-negotiable. Any skill that modifies files, runs commands, or interacts with external services should include safety checks.

Here is a safety checklist to consider for every skill:

Risk	Safety Measure	Example
Accidentally overwriting files	Ask before writing	"Show the output and ask the user to confirm before saving"
Deleting important data	List what will be deleted first	"Show the list of files to be removed. Do NOT delete without explicit confirmation"
Exposing secrets	Check for sensitive patterns	"Scan for API keys, passwords, and tokens before including in output"
Running destructive commands	Dry-run first	"Run with --dry-run flag first and show what would happen"
Infinite loops or long operations	Set limits	"Process a maximum of 50 files. If more exist, ask the user to narrow the scope"

Here is a real example of safety rails in a file cleanup skill:

```
## Safety Rules (MUST follow these)
- NEVER delete files without showing the full list first and getting
  explicit user confirmation
- NEVER modify files outside the current project directory
- If more than 20 files would be affected, stop and ask the user
  to narrow the scope
- Create a backup note listing all affected files before making changes
```

Putting the safety rules in their own section with strong language ("NEVER," "MUST") helps ensure Claude Code treats them as hard constraints rather than suggestions.

Principle 5: Handle Edge Cases Gracefully

What happens when your changelog skill runs on a project with zero commits? What happens when your TODO finder searches a project with no source files? What happens when the user invokes your skill from the wrong directory?

Great skills anticipate these situations:

```
## Edge Cases
- If there are no staged changes, inform the user:
  "No files are staged for commit. Use `git add <file>` to stage
  changes first."
- If the project has no git history, inform the user:
  "This directory is not a git repository. Run `git init` first."
- If the diff is extremely large (more than 500 lines), summarise at a
  high level and ask if the user wants the full detailed analysis.
```

Think of it like defensive driving. You are not expecting an accident, but you are prepared for one. Your skill should always produce helpful output, even when conditions are not ideal.

Principle 6: Write for Your Future Self (and Your Team)

Skills are code. Well, they are Markdown files, but they serve the same purpose as code—they encode logic that gets executed. And just like code, they need to be understandable six months from now when you have forgotten why you wrote them.

Add context comments:

```
---
name: deploy-check
description: Pre-deployment checklist for production releases
---

# Deploy Check Skill
# Created: 2026-02-15
# Author: Your Name
# Purpose: Runs through our team's deployment checklist before pushing
#          to production. Based on the incident post-mortem from January
#          where we forgot to update environment variables.

## Step 1: Verify Build
...
```

That context comment at the top costs you 30 seconds to write and will save you (or a teammate) 30 minutes of confusion later.

Use descriptive names:

Bad Name	Good Name	Why
check.md	pre-deploy-check.md	Clear what kind of check
helper.md	component-scaffold.md	Descriptive of the actual task
review.md	security-review.md	Distinguishes from code review
do.md	run-test-suite.md	Specific about what it does

Principle 7: Project Skills vs Personal Skills—Choose Wisely

We touched on this in the previous section, but let's go deeper on when to use each.

Use project skills (.claude/skills/) when:

- The skill enforces team conventions (commit message format, code style)
- The skill references project-specific paths, configs, or patterns
- Multiple people work on the project and should follow the same workflow
- The skill is part of the development process (like a pre-commit check)

Use personal skills (~/.claude/skills/) when:

- The skill reflects your personal preferences (how you like comments formatted)
- The skill is useful across multiple unrelated projects
- The skill contains your personal productivity hacks
- You are experimenting and do not want to clutter the team's skill directory

Version control considerations:

- Project skills should be committed to Git. They are part of the project's development infrastructure.
- Personal skills should not be committed to any project repo (they live in your home directory, which is outside the project).
- Add a note in your project's README if you add skills: "This project includes Claude Code skills in .claude/skills/. See the files for available slash commands."

Principle 8: Test with Different Scenarios

Before you consider a skill "done," test it against at least three scenarios:

1. The happy path — Everything is normal. The project has commits, files exist, commands work. Does the skill produce good output?
2. The empty case — No commits, no files, no staged changes. Does the skill handle this gracefully with a helpful message?
3. The overload case — Hundreds of commits, thousands of files, a massive diff. Does the skill handle the volume without timing out or producing an unreadable wall of text?

Here is a simple testing checklist:

```
Skill Testing Checklist:
&  Test with a normal project (5-20 files, some git history)
&  Test with an empty/new project
&  Test with a large project (100+ files)
&  Test with unexpected input (wrong arguments, weird file names)
&  Test the safety rails (does it actually stop when it should?)
&  Test with a teammate (do they understand the output?)
```

Principle 9: Evolve Your Skills Over Time

Your skills should grow with you. Here is a healthy evolution pattern:

Week 1: Basic skill that gets the job done.

```
Run tests and tell me if they pass.
```

Week 3: Add structure and error handling.

```
## Step 1: Identify Test Framework
Check package.json for jest, vitest, mocha, or other test runners.

## Step 2: Run Tests
Execute the appropriate test command.

## Step 3: Analyse Results
Parse the output and summarise: passed, failed, skipped.
Highlight any failures with the test name and error message.
```

Month 2: Add advanced features.

```
## Step 1: Identify Test Framework
...

## Step 2: Check for Related Tests
If the user has staged changes, find test files related to the
modified source files. Suggest running just those tests first.

## Step 3: Run Tests
...

## Step 4: Coverage Analysis
If coverage reporting is available, show a summary of coverage
for the modified files.

## Step 5: Suggest Missing Tests
For any modified files without corresponding test files,
suggest what tests should be written.
```

Do not try to build the Month 2 version on Day 1. Start simple, use the skill daily, notice what is missing, and add it.

Principle 10: Learn from the Community

As Claude Code's skill ecosystem grows, pay attention to how others write their skills. Look for patterns like:

- Output formatting conventions — How do popular skills present their results?
- Safety patterns — What confirmation mechanisms do others use?
- Tool usage patterns — Which Claude Code tools (Bash, Grep, Read, Edit) are most effective for which tasks?
- Error handling idioms — How do well-designed skills handle failures?

When you find a skill pattern you like, adapt it for your own use. The best skill authors are not starting from zero—they are remixing and improving ideas from the community.

Quick Reference: Skill Design Checklist

Use this checklist every time you create or update a skill:

```
Skill Design Checklist:
&  Does the skill do ONE thing well?
&  Are commands and tools specified explicitly?
&  Are instructions structured with headings and steps?
&  Are safety rails in place for any destructive actions?
&  Are edge cases handled with helpful messages?
&  Is there context for future readers (comments, dates, purpose)?
&  Is the file name descriptive?
&  Is it in the right location (project vs personal)?
&  Has it been tested with happy path, empty case, and overload?
&  Is it committed to Git (if it is a project skill)?
```

Print this out, stick it next to your monitor, and reference it every time you create a skill. After a few weeks, these principles will become second nature.

A Note for Cursor and Other Tool Users

While this section focuses on Claude Code skills, the principles apply broadly to any AI coding tool:

- Cursor's .cursorrules benefits from the same clarity, specificity, and structure we discussed.
- GitHub Copilot's custom instructions work better when they are scoped, specific, and safety-conscious.
- Lovable's AI instructions produce more consistent results when you structure them with clear steps.

The mental model is universal: clear instructions produce consistent results. The file format varies, but the thinking is the same.

Key Takeaways

- Scope skills tightly—one skill, one job. If you need the word "and" to describe it, split it.
- Be specific about commands and tools—exact commands beat vague instructions every time.
- Use Markdown structure (headings, lists, code blocks) to make instructions clear and sequential.
- Always include safety rails—confirm before writing, warn before deleting, check before exposing secrets.
- Handle edge cases gracefully—empty projects, no git history, large diffs. Your skill should always produce helpful output.
- Write for your future self—add context comments, use descriptive names, and document the "why" behind design choices.

- Choose project vs personal skills deliberately—team conventions go in the project, personal preferences go in your home directory.
- Test against three scenarios: normal, empty, and overloaded.
- Evolve over time—start simple, use daily, improve incrementally.
- These principles apply across all AI coding tools, not just Claude Code.

Real-World Skill Examples

You know that feeling when you walk into a well-organised kitchen and everything is labelled, every tool has its place, and there is a recipe card pinned above the counter for every dish the household makes regularly? That is what a project with a solid set of custom skills feels like. Everything has a shortcut. Every repetitive task has a one-command solution.

In this section, we are going to build five complete, production-ready skills that you can drop into your project today. These are not toy examples but rather they are based on real workflows that developers and non-coders use daily. For each one, I will show you the full skill file, explain why it is designed the way it is, and give you tips for customising it to your own needs.

Skill 1: The Commit Message Generator

What it does: Analyses your staged changes and generates a well-formatted commit message following the Conventional Commits standard.

Why you need it: Commit messages are the diary of your project. Six months from now, when something breaks, you will be searching through commit history trying to find when a change was introduced. Good commit messages are the difference between "found it in 5 minutes" and "gave up after an hour."

The full skill file (.claude/skills/commit.md):

```
---
name: commit
description: Generate a conventional commit message from staged changes
---

# Commit Message Generator

Analyse staged git changes and generate a high-quality commit message
following the Conventional Commits specification.

## Step 1: Check for Staged Changes

Run `git diff --cached --stat` to verify there are staged changes.

If nothing is staged:
- Run `git status --short` to show the current state
- Tell the user: "No changes are staged. Use `git add <file>` to stage
  files before committing."
- Stop here.

## Step 2: Analyse the Changes
```

<type>(<scope>): <subject>

Types:

- feat: New feature or functionality
- fix: Bug fix
- docs: Documentation changes only
- style: Code style changes (formatting, semicolons, etc.)
- refactor: Code restructuring without changing behaviour
- test: Adding or updating tests
- chore: Maintenance tasks (deps, configs, build)

Rules:

- Subject line: imperative mood, lowercase, no period, under 72 chars
- Body: explain WHY the change was made, not just WHAT changed
- Footer: reference issue numbers if apparent (e.g., "Closes #42")
- If changes span multiple concerns, suggest splitting into separate commits

Step 5: Present and Confirm

Show the generated commit message in a code block.

Ask the user:

1. "Commit with this message?" (proceed)
2. "Edit the message?" (let them modify it)
3. "Cancel" (abort without committing)

Only run git commit -m "<message>" after explicit confirmation.
If the message has a body, use the multi-line format:
git commit -m "<subject>" -m "<body>"

Why this skill is well-designed:
- It checks for staged changes before doing anything (edge case handling)
- It scans for secrets before committing (safety rail)
- It suggests splitting if changes are too broad (quality enforcement)
- It asks for confirmation before running the actual commit (user control)

Customisation ideas:
- Add your team's ticket number format (e.g., "Always reference JIRA tickets in the footer")
- Include a co-author line if you pair program
- Add project-specific scopes (e.g., "Use scopes: api, ui, db, auth, config")

Skill 2: The Code Reviewer

What it does: Performs a thorough code review of changed files, checking for bugs, secur...

Why you need it: Code review is one of the most valuable practices in software developme...

The full skill file (`.claude/skills/review.md`):

```markdown
---
name: review
description: Perform a thorough code review of recent changes
---

# Code Review Skill

Review changed files for bugs, security issues, performance concerns, and
code quality. Provide actionable feedback in a structured format.

## Step 1: Identify What to Review

Check what has changed:
1. First, check for staged changes: `git diff --cached --name-only`
2. If nothing is staged, check unstaged changes: `git diff --name-only`
3. If the user provides a branch name or commit range, use that instead

List the files that will be reviewed and their change type
(modified, added, deleted).

## Step 2: Read and Analyse Each File

For each changed file, read the full diff and analyse it against
these categories:

### Bugs and Logic Errors
- Off-by-one errors
- Null/undefined reference risks
- Missing error handling (try/catch, .catch() on promises)
- Race conditions or async issues
```

Code Review Summary

Files reviewed: 5
Issues found: 3 critical, 5 suggestions, 2 nits

Critical Issues (must fix)

1. [filename:line] Description of the issue

Suggestions (should fix)

1. [filename:line] Description

Nits (nice to have)

1. [filename:line] Description

What Looks Good

- Positive observations about the code

```
Rules:
- Be constructive, not critical. Frame issues as suggestions.
- Always explain WHY something is a problem, not just WHAT is wrong.
- Include at least one positive observation (what looks good).
- If no issues are found, say so clearly and note what was checked.
- Prioritise critical issues over nits.

## Step 4: Offer to Help Fix

After presenting the review, ask:
"Would you like me to help fix any of these issues?"

If the user says yes, address the critical issues first.
```

Why this skill is well-designed:

- It has a clear checklist-based approach (nothing gets forgotten)
- It categorises issues by severity (helps prioritise)
- It includes positive feedback (not just criticism)
- It offers to help fix issues (goes beyond just pointing out problems)

Customisation ideas:

- Add project-specific patterns to check for (e.g., "Ensure all API routes use the auth middleware")
- Include framework-specific checks (e.g., React hooks rules, Next.js server vs client components)
- Add your team's style guide rules

Skill 3: The Test Writer

What it does: Analyses a source file and generates comprehensive test cases for it, using the appropriate testing framework.

Why you need it: Writing tests is one of those things everyone knows they should do but often skip because it feels tedious. This skill removes the friction by generating a solid starting point for your test suite.

The full skill file (.claude/skills/write-tests.md):

```
---
name: write-tests
description: Generate test cases for a source file
---

# Test Writer Skill

Generate comprehensive test cases for a given source file. Detect the
testing framework and write tests that cover the main functionality.

## Step 1: Identify the Target

If the user specifies a file, use that file.
If no file is specified:
- Check for staged changes: `git diff --cached --name-only`
- Filter to source files (exclude test files, configs, styles)
- Ask the user which file to write tests for

## Step 2: Detect the Testing Setup

Read `package.json` to determine:
- Testing framework: Jest, Vitest, Mocha, or Playwright
- Assertion library: built-in, Chai, or Testing Library
- Whether React Testing Library is available (for component tests)
- The test file naming convention (*.test.ts, *.spec.ts, etc.)
- The test directory structure (co-located or separate __tests__ folder)

If no testing framework is found:
- Suggest installing one (recommend Vitest for Vite projects, Jest otherwise)
- Provide the install command
- Stop until the user sets up testing

## Step 3: Analyse the Source File

Read the target file and identify:
- Exported functions and their parameters
- Component props and their types (for React components)
- Side effects (API calls, database queries, file operations)
- Edge cases (null inputs, empty arrays, boundary values)
```

Why this skill is well-designed:

- It auto-detects the testing framework (no configuration needed)
- It handles different types of code (utils, components, API routes)
- It generates meaningful test names (not just "test 1, test 2")
- It respects the project's existing test conventions

Customisation ideas:

- Add minimum coverage targets (e.g., "Generate enough tests to cover at least 80% of branches")
- Include snapshot testing for components
- Add performance test templates for critical paths

Skill 4: The Documentation Generator

What it does: Reads source files and generates clear, structured documentation including function descriptions, parameter types, return values, and usage examples.

Why you need it: Documentation is the gift you give your future self (and your teammates). But writing it manually is a chore that usually falls to the bottom of the priority list. This skill automates the tedious parts while still producing documentation humans actually want to read.

The full skill file (.claude/skills/generate-docs.md):

```
---
name: generate-docs
description: Generate documentation for source files or the entire project
---

# Documentation Generator Skill

Generate clear, structured documentation for source code files. Support
both individual file documentation and project-wide overviews.

## Step 1: Determine Scope

If the user specifies a file or directory, document that.
If no target is specified, ask:
- "Document a specific file?" (focused docs)
- "Generate a project overview?" (README-style)
- "Document all exports in a directory?" (API reference)

## Step 2: Analyse the Code

For each file to be documented, read it and identify:
- Module purpose (what does this file do?)
```

Notes:

- Important behaviour to be aware of
- Edge cases or limitations

````
### For React Components
```markdown
### `<ComponentName />`

Brief description of the component.

**Props:**
| Prop | Type | Required | Default | Description |
|------|------|----------|---------|-------------|
| title | string | Yes | - | The heading text |
| variant | 'primary' \| 'secondary' | No | 'primary' | Visual style |

**Usage:**
```tsx
<ComponentName title="Hello World" variant="secondary" />
````

Notes:

- Accessibility considerations
- Responsive behaviour

````
### For Project Overview
Generate a README-style document covering:
- Project purpose and description
- Tech stack summary
- Directory structure with descriptions
- Setup instructions (based on package.json scripts)
- Available commands
- Environment variables needed (from .env.example if it exists)

## Step 4: Format and Style Rules

- Use clear, jargon-free language
- Include at least one code example per function/component
- Use tables for parameters and props
- Add a "Quick Start" section at the top of project overviews
- Cross-reference related functions ("See also: relatedFunction()")
- Include the file path at the top of each file's documentation

## Step 5: Present and Save

Show the generated documentation to the user.

Ask where to save it:
````

Why this skill is well-designed:

- It handles multiple documentation scopes (file, directory, project)
- It uses tables for structured information (easy to scan)
- It includes code examples (documentation without examples is barely useful)
- It offers multiple output formats (separate file vs inline comments)

Customisation ideas:

- Add your project's documentation template or style guide
- Include changelog generation alongside docs
- Add API endpoint documentation format for backend routes

Skill 5: The Deployment Checklist

What it does: Runs through a comprehensive pre-deployment checklist, verifying that your project is ready for production.

Why you need it: Deployments are where things go wrong. You forget to update an environment variable. You leave console.log statements in production code. You skip the build step and push broken assets. This skill is your co-pilot during the most nerve-wracking part of the development process.

The full skill file (.claude/skills/deploy-check.md):

```
---
name: deploy-check
description: Run a pre-deployment checklist to verify production readiness
---

# Pre-Deployment Checklist

Run through a comprehensive checklist before deploying to production.
Report each check as PASS, FAIL, or WARN with actionable details.

## Check 1: Clean Working Directory

Run `git status` to verify:
- No uncommitted changes
- No untracked files that should be committed
- Currently on the correct branch (main or release branch)

If there are uncommitted changes, WARN and ask if the user wants to
stash or commit them first.

## Check 2: Build Succeeds
```

Deployment Readiness Report

Check	Status	Details
Clean working directory	PASS	
Build succeeds	PASS	
No console statements	WARN	3 found in /src
Environment variables	PASS	
Dependencies secure	PASS	
Tests pass	FAIL	2 failures
TypeScript clean	PASS	

Overall: NOT READY (1 FAIL, 1 WARN)

```
If all checks pass: "Your project is ready for deployment!"
If any checks fail: "Please fix the FAIL items before deploying."
If only warnings: "Your project can be deployed, but review the
WARN items when possible."

Ask: "Would you like help fixing any of the issues found?"
```

Why this skill is well-designed:

- It uses a clear PASS/FAIL/WARN system (immediately scannable)
- It checks multiple dimensions (code quality, security, build, tests)
- It stops on critical failures (does not let you deploy broken code)
- It offers to help fix issues (not just a report, but a helper)

Customisation ideas:

- Add project-specific checks (e.g., "Verify Stripe keys are set to live mode")
- Include database migration checks ("Are there pending migrations?")
- Add performance budget checks ("Is the bundle size under 500KB?")
- Include a deployment command at the end if all checks pass

Putting It All Together: A Day in the Life

Here is how these five skills might fit into a typical development day:

Morning: Start working on a feature

1. Write your code, make changes across several files
2. /review — Get AI feedback on your changes before committing

3. Fix any issues the review identified

Midday: Ready to commit
4. Stage your changes with git add
5. /commit — Generate a clean, conventional commit message
6. Push to your branch

Afternoon: Write tests and docs
7. /write-tests — Generate test cases for the new code
8. Run the tests, fix any that fail
9. /generate-docs — Update documentation for changed files

End of day: Deploy
10. /deploy-check — Run the full pre-deployment checklist
11. Fix any failures or warnings
12. Deploy with confidence

That is five slash commands replacing what used to be hours of manual, error-prone work. And the best part? The quality is consistent. Your Friday afternoon deploy gets the same thorough checklist as your Monday morning one.

How to Install These Skills in Your Project

If you want to use all five skills from this section, here is the quick setup:

```
# Create the skills directory
mkdir -p .claude/skills

# Create each skill file
touch .claude/skills/commit.md
touch .claude/skills/review.md
touch .claude/skills/write-tests.md
touch .claude/skills/generate-docs.md
touch .claude/skills/deploy-check.md
```

Then copy the content from each skill example above into the corresponding file. Or even better, ask Claude Code to do it:

```
Create the five skill files in .claude/skills/ with the content from
the course examples: commit, review, write-tests, generate-docs, and
deploy-check.
```

Yes, you can use AI to set up your AI skills. We are truly living in the future.

Adapting Skills for Different AI Tools

While these skill files are written for Claude Code, the logic inside them translates to other tools:

Skill	Claude Code	Cursor	Lovable / Replit
Commit generator	.claude/skills/commit.md	Add commit rules to .cursorrules	Include in project AI instructions
Code reviewer	.claude/skills/review.md	Use as a Notepad snippet attached to review chats	Paste as instructions when asking for review
Test writer	.claude/skills/write-tests.md	Reference in .cursorrules test section	Include testing instructions in project settings
Doc generator	.claude/skills/generate-docs.md	Use as Notepad for doc-related tasks	Add to project AI instructions
Deploy checklist	.claude/skills/deploy-check.md	Add pre-deploy rules to .cursorrules	Create a checklist prompt template

The key insight is that the hard work is in designing the instructions, not in the file format. Once you have a great set of instructions, adapting them to any AI tool is straightforward.

Key Takeaways

- Five production-ready skills covered: commit messages, code review, test writing, documentation generation, and deployment checklists.
- Each skill follows the best practices from the previous section: focused scope, specific commands, structured steps, safety rails, and edge case handling.
- Skills integrate into your daily workflow naturally from morning coding to end-of-day deployment.
- Customise these examples for your specific project, add your team's conventions, your tech stack's quirks, and your deployment target's requirements.
- The instructions translate across AI tools—once you have good skill logic, you can adapt it for Cursor, Lovable, Replit, or any other AI coding assistant.
- Start with one or two skills that solve your biggest pain points, then add more as you get comfortable with the pattern.
- Share your skills with your team by committing them to the project repo, everyone benefits from consistent, automated workflows.